

Detailed Fix Direction: Tile Engine → Dispatcher Migration

Author: Muhammed Öztürk

Date: 2026-06-03

Status: Pre-implementation — for team review

1. Why This Project Exists

1.1 The Duplication Problem

Today, CK-Tile has two separate systems that both do the same thing: generate and run GEMM kernels.

Tile Engine (TE):

- Manages kernel generation via CMake
- Each config JSON → Python instance builder → many C++ generated files
- Each config compiles into its own binary (benchmark_gemm_fp16_rcr_compv3_...)
- Hundreds of binaries, one per (dtype, layout, pipeline, tile-shape) combination
- Validation and benchmarking embedded in these binaries

Dispatcher:

- Manages kernel generation via `unified_gemm_codegen.py` (Python, already exists)
- All kernels compile into one shared library (`libdispatcher_gemm_lib.so`)
- Python interface (`DispatcherLib`) dispatches to the right kernel at runtime
- One `.so`, one interface — hardware-aware selection happens in Python/C++

Having both systems creates:

- Redundant C/C++ generated files (same kernel described twice)
- Scaling problems (hundreds of binaries instead of one library)
- Maintenance burden (changes must be made in two places)
- Build time waste (compiling the same kernels twice)

1.2 Vidyasagar's Principle

From the team meeting (2026-06-03):

Do NOT make the Dispatcher look like Tile Engine.

Make Tile Engine conform to the Dispatcher model.

This is the key architectural direction. It means:

- Dispatcher's vocabulary is canonical (`wave_m`, `auto`, `row/col`)

- Dispatcher's Python interface (DispatcherLib) is the production entry point
 - TE's vocabulary (warp_m, default, rcr) adapts to the Dispatcher, not vice versa
-

2. What "Replace" Means Concretely

2.1 What gets deleted

```
tile_engine/ops/gemm/gemm_universal/  
  gemm_universal_instance_builder.py ← replaced by dispatcher codegen  
  CMakeLists.txt (kernel gen rules) ← replaced by Python driver  
  
tile_engine/ops/gemm/  
  bin/benchmark_gemm_fp16_rcr_* ← replaced by DispatcherLib.run_gemm()  
  bin/benchmark_gemm_bf16_rcr_*  
  ... (hundreds of binaries)  
  
test/ck_tile/gemm_tile_engine/ ← replaced by dispatcher parity tests
```

2.2 What remains

```
dispatcher/codegen/  
  unified_gemm_codegen.py ← stays, this IS the replacement codegen  
  arch_specs.json ← stays  
  
dispatcher/python/  
  ctypes_utils.py ← stays, this IS the replacement interface  
  DispatcherLib ← the production entry point  
  GemmRunner ← helper for numpy-based runs  
  Registry ← JIT compilation manager  
  
dispatcher/parity/  
  te_to_dispatcher.py ← stays, used during migration verification  
  check_identifier_parity.py ← stays, regression guard  
  check_parity.py ← stays, extended to use DispatcherLib  
  drive_codegen.py ← stays, core driver mechanism
```

2.3 What the thin frontend looks like

After migration, anything that used to call a TE binary:

```
# OLD: TE approach  
./benchmark_gemm_fp16_rcr_compv3_default_intrawave_..._256x256x32 \  
  -m=1024 -n=1024 -k=1024 -warmup=50 -repeat=100
```

Now calls:

```
# NEW: Dispatcher approach  
from ctypes_utils import DispatcherLib  
import numpy as np  
  
lib = DispatcherLib.load()  
lib.initialize()  
  
M, N, K = 1024, 1024, 1024
```

```

A = np.random.randn(M, K).astype(np.float16)
B = np.random.randn(K, N).astype(np.float16)
C = np.zeros((M, N), dtype=np.float16)

status, time_ms = lib.run_gemm(A, B, C, M, N, K)
tflops = 2 * M * N * K / (time_ms * 1e-3) / 1e12
print(f"{time_ms:.3f} ms  {tflops:.2f} TFLOPS")

```

No binary name to construct. No CMake needed. One library, one Python call.

3. What the Current PR Built vs What Is Needed

3.1 Current PR architecture

```

TE JSON config
  ↓
te_to_dispatcher.py  (translator)
  ↓
drive_codegen.py     (calls unified_gemm_codegen.py as subprocess)
  ↓
gemm_*.hpp           (generated header)
  ↓
harness.cpp          (custom single-kernel C++ runner)
  ↓
compare output vs TE binary

```

3.2 Problems with current PR

Problem A — Translator makes Dispatcher speak TE's language.

`te_to_dispatcher.py` takes TE JSON as input. TE vocabulary is still the source of truth. The Dispatcher adapts to TE — the opposite of what Vidyasagar said.

Problem B — Custom harness bypasses the production interface.

`harness.cpp` is a one-off C++ runner. Production callers will use `DispatcherLib.run_gemm()`. Parity proved via `harness` does not prove that `DispatcherLib` works. These are different code paths.

Problem C — `drive_codegen.py` invokes `codegen` outside the build system.

`unified_gemm_codegen.py` is already wired into CMake. `drive_codegen.py` calls it again as a subprocess — creating a parallel, ad-hoc build path outside CMake. This is fine for verification but should not be the production model.

3.3 What needs to change

Current	Target
Parity via <code>harness.cpp</code>	Parity via <code>DispatcherLib.run_gemm()</code>
TE JSON as primary input	Dispatcher config format as primary input

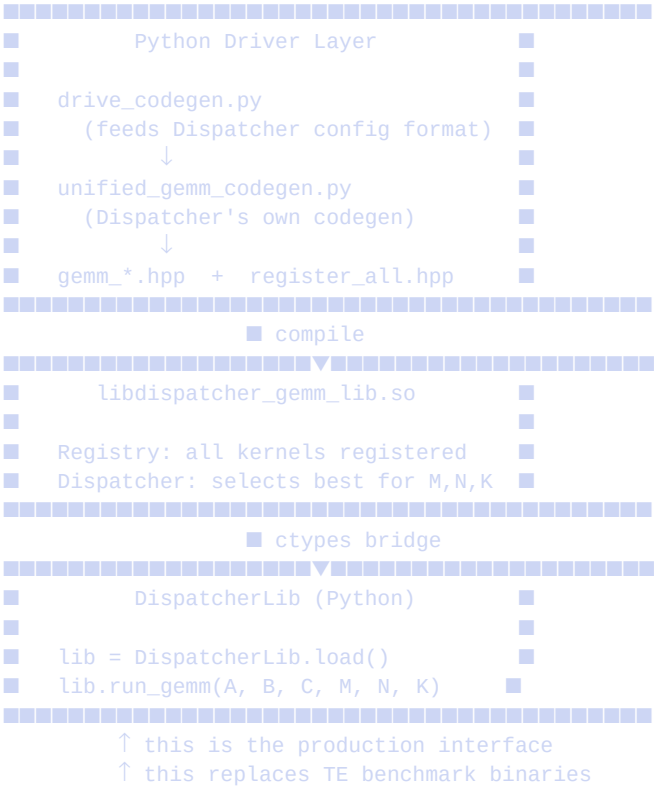
Current	Target
Custom C++ runner	Existing Python interface
Proves: custom harness $\approx$ TE	Proves: DispatcherLib $\approx$ TE binary

3.4 What to keep from current PR

The current PR's code is not wasted — it is useful for:

- `te_to_dispatcher.py` — field mapping table that documents every TE→Dispatcher translation. Invaluable reference during migration.
- `check_identifier_parity.py` + 220 tests — proves codegen and C++ runtime produce identical kernel names. This is a critical regression guard and should stay in the test suite permanently.
- `drive_codegen.py` — the mechanism for driving codegen for specific configs. Refactor so it produces headers that DispatcherLib loads, not headers for a custom harness.
- `check_parity.py` — the 3-stage parity orchestrator. Extend Stage 2 to use `DispatcherLib.run_gemm()` instead of a compiled harness binary.

4. Correct Architecture After Migration



The parity test compares:

TE binary (-m=M -n=N -k=K)	↔	DispatcherLib.run_gemm(A,B,C,M,N,K)
numerical output C		numerical output C
TFLOPS reported		TFLOPS computed from time_ms

5. Phase 1: End-to-End for One Config

5.1 Choose a target config

Recommended: fp16 / rcr / compv3 / intrawave / no-padding

This is the most common config in CI and the simplest to debug.

5.2 Step-by-step

Step 1 — Confirm unified_gemm_codegen.py **generates the right header**

```
# drive_codegen.py already does this – verify it still works
cfg = {
    "dtype_a": "fp16", "dtype_b": "fp16", "dtype_c": "fp16",
    "layout_a": "row", "layout_b": "col", "layout_c": "row",
    "tile_m": 256, "tile_n": 256, "tile_k": 32,
    "wave_m": 2, "wave_n": 2, "wave_k": 1,
    "warp_m": 32, "warp_n": 32, "warp_k": 8,
    "pipeline": "compv3", "scheduler": "auto",
    "epilogue": "default", "gfx_arch": "gfx942",
    "pad_m": False, "pad_n": False, "pad_k": False,
    "persistent": False,
}
```

Step 2 — Build the .so with that kernel

CMake already handles this. Confirm it picks up the new header.

If building outside CMake: drive_codegen.py + hipcc to produce the .so.

Step 3 — Load via DispatcherLib and run

```
from ctypes_utils import DispatcherLib
import numpy as np

lib = DispatcherLib.load()
assert lib is not None, "Build the .so first"
lib.initialize()

M, N, K = 1024, 1024, 1024
A = (np.random.randn(M, K) * 0.1).astype(np.float16)
B = (np.random.randn(K, N) * 0.1).astype(np.float16)
C_dispatcher = np.zeros((M, N), dtype=np.float16)

status, time_ms = lib.run_gemm(A, B, C_dispatcher, M, N, K)
assert status == 0, f"run_gemm failed with status {status}"
```

Step 4 — Run the same problem through TE binary

```
./benchmark_gemm_fp16_rcr_compv3_default_intrawave..._256x256x32 \
-m=1024 -n=1024 -k=1024 -verify=1 -json_output=true > te_result.json
```

Extract C tensor from TE output.

Step 5 — Compare

```
# Numerical parity
C_ref = np.matmul(A.astype(np.float32), B.astype(np.float32)).astype(np.float16)
max_err = float(np.max(np.abs(C_dispatcher - C_ref)))
assert max_err < 1e-2, f"Numerical mismatch: {max_err}"

# Performance parity
tflops_dispatcher = 2 * M * N * K / (time_ms * 1e-3) / 1e12
# Compare against TE reported TFLOPS – within 2%
```

Phase 1 done when: both assertions pass on fp16/rcr/compv3 at 1024×1024×1024.

6. Phase 2: Full Sweep and TE Deletion

6.1 Batch translation (T2.1)

Extend `te_to_dispatcher.py` to accept a full TE config file (not just one row) and output a list of Dispatcher configs. Also produce a rejection manifest:

```
configs/fp16_rcr_compv3_all.json  ← all valid Dispatcher configs
configs/rejected.csv              ← combos TE has but Dispatcher rejects, with reason
```

6.2 Multi-kernel sweep (T2.3)

```
# sweep_runner.py – already exists in current PR, extend it
for config in all_configs:
    for M, N, K in problem_sizes:
        status, time_ms = lib.run_gemm(A, B, C, M, N, K)
        record(config, M, N, K, status, time_ms, numerical_error)
```

Output: Parquet file with one row per (config, problem).

6.3 Coverage targets (T2.4, T2.5)

Dtype	Layout	Target pass rate	TFLOPS delta
fp16	rcr	≥99%	≤2%
bf16	rcr	≥99%	≤2%
fp8	rcr	≥99%	≤5% (fp8 has wider variance)

6.4 Comparison report (T2.6)

One-command script that reads both Parquet files and produces:

```
report.html
```

- Summary table: dtype × layout × pass rate × median TFLOPS delta
- Per-config detail: TE time vs Dispatcher time vs delta vs status
- Rejection log: what TE had that Dispatcher skipped, and why

6.5 Deletion (final step)

After ≥99% pass rate confirmed:

```
git rm tile_engine/ops/gemm/gemm_universal/CMakeLists.txt
git rm tile_engine/ops/gemm/gemm_universal/gemm_universal_instance_builder.py
git rm -r tile_engine/ops/gemm/gemm_universal/generated/
# update CI scripts to call DispatcherLib instead of TE binaries
```

7. Naming Traps to Avoid

These are the bugs that look like numerical errors but are actually config errors.

7.1 warp vs wave (the most dangerous one)

TE field	Meaning	Dispatcher field
warp_m=2	2 waves per block in M dim	wave_m=2
warp_tile_m=32	MFMA tile size in M dim	warp_m=32

If you swap these, you get a kernel that compiles and runs but produces wrong answers silently. `te_to_dispatcher.py` already handles this correctly — do not re-implement this mapping anywhere else.

7.2 scheduler "default" vs "auto"

- TE uses "default" in its kernel names
- Dispatcher's `to_string(Scheduler::Auto)` returns "auto"
- The registry lookup uses the Dispatcher string
- `te_to_dispatcher.py` maps "default" → "auto" — keep this mapping

7.3 Output dtype for fp8

TE: fp8 input → fp16 output (hardcoded in TE)

Dispatcher: fp8 input → fp32 accumulator → configurable output

The translation must set `dtype_c = "fp16"` for fp8 inputs to match TE behavior.

8. Risk Register

Risk	Likelihood	Impact	Mitigation
Dispatcher .so not built on available machine	High	Blocks Phase 1	Confirm build environment first; Phase 1 identifier tests run without GPU
DispatcherLib.run_gemm() dtype hardcoded in .so	Medium	Blocks multi-dtype	Check gemm_ctypes_lib.cpp; may need dtype flag in C API
TE configs that Dispatcher rejects	Medium	Coverage gap	Use rejection manifest; consult Vidyasagar per case
AI agents diverge from requirements again	High	Wasted effort	Human reviews plan before each PR; no code without team sign-off

9. Immediate Next Steps

- **Team reviews this document** — Vidyasagar, Thrupti, Yaswanth confirm the direction is correct before any code is written.

- **Confirm build environment** — Is the Dispatcher .so built and accessible? Which machine has GPU + ROCm for Phase 1 numerical tests?

- **Confirm target config** — Which single config is the Phase 1 target?

- **Write Phase 1 parity script** — parity_dispatcherlib.py that uses DispatcherLib.run_gemm() (not harness.cpp) as the Dispatcher side.

- **Update PR description** — Replace current PR scope with corrected direction. Existing code (translator, identifier tests, codegen driver) is kept as-is. Only the parity proof method changes (harness → DispatcherLib).

10. One-Page Summary for the Team

	Current (wrong)	Target (correct)
Parity proof via	Custom harness.cpp	DispatcherLib.run_gemm()
Primary input	TE JSON	Dispatcher config format
Codegen invocation	drive_codegen.py subprocess	CMake build (canonical)
Runtime interface	Ad-hoc harness binary	DispatcherLib (production Python)
End state	TE + Dispatcher + harness all running	TE deleted, DispatcherLib is the only path
Architectural direction	Dispatcher speaks TE's language	TE conforms to Dispatcher model

The change to make in the current PR: replace the harness.cpp parity step with a DispatcherLib.run_gemm() parity step. Everything else (translator, identifier

tests, codegen driver) stays and is useful.